2주차

Google Developer Group Hongik University

백엔드 정규 스터디 2주차

2025.10.02 (2주차)
계층형 아키텍처와 컨트롤러, 서비스

Kun Lee 혼돈균

GDG Hongik Co-Organizer | leekun775@gmail.com

2주차

스터디 소개

- 백엔드 & 스프링 입문자를 위한 6주 과정
- 온라인 쇼핑몰 프로젝트를 직접 구현하며 스프링 핵심 개념 학습
- **탑다운(Top-down) 방식**으로 백엔드 개발의 전체적인 흐름을 쉽게 이해
- 배경 지식 - Java, 객체지향, 기초 SQL
- 매주 수요일 업로드 되는 강의를 듣고 과제 제출

2주차

스터디 소개

1주차 (9/24) - 웹과 HTTP, REST API

2주차 (10/02) - 계층형 아키텍처와 컨트롤러, 서비스

--------- 추석 연휴 & 중간고사 ---------

3주차 (10/29) - JPA와 레포지토리

4주차 (11/5) - ERD, DB, 엔티티

5주차 (11/12) - SOLID 원칙과 스프링 빈

6주차 (11/19) - 예외 처리와 Swagger

2주차

Index

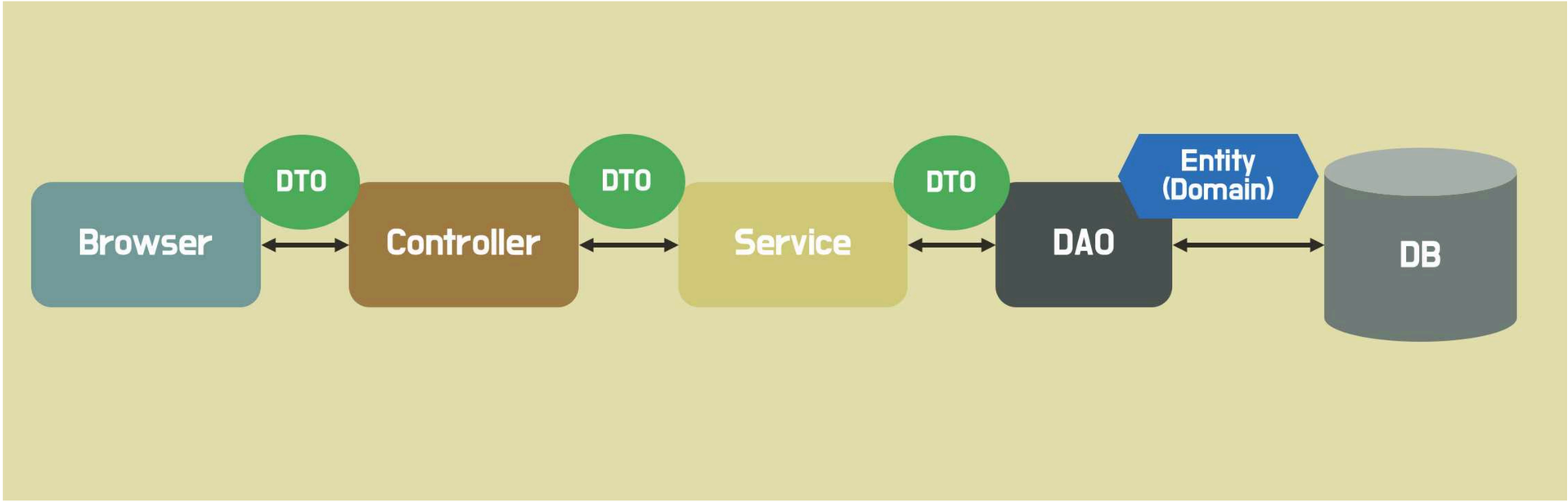
1. **Spring의 핵심 설계:** 계층형 아키텍처(Layered Architecture)
2. **웹 요청의 진입점:** 컨트롤러 계층(Controller Layer)
3. **스프링 빈 & 의존성 주입 (Bean & DI) - 압축 ver.**
4. **비즈니스 로직의 중심:** 서비스 계층(Service Layer)

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

스프링 계층형 아키텍처(Layered Architecture)?

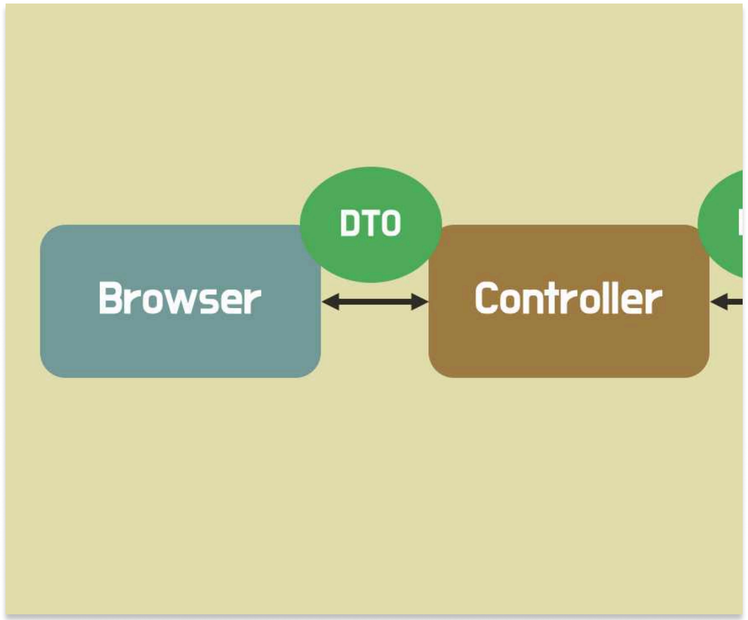


2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Layered Architecture = 레스토랑



Browser(Frontend)
= 손님

Controller
= 웨이터

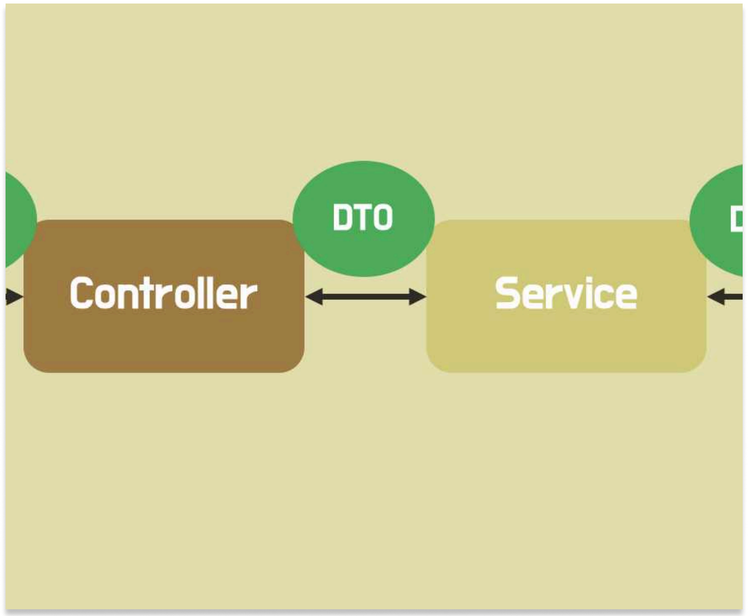
Browser → Controller: 스텍크 내놔@!

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Layered Architecture = 레스토랑



Controller(컨트롤러 계층)

- Browser와 소통
- 고기 굽는 법 몰라 (비즈니스 로직 x)
- Service한테 시킴.

Service
= 주방장

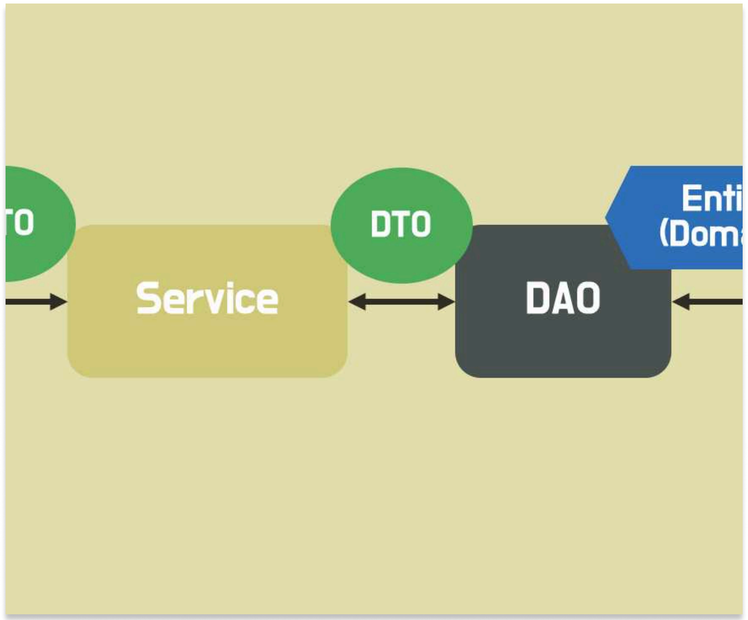
Controller → Service: 스텍크 내놔!!

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Layered Architecture = 레스토랑



Service(서비스 계층)

- 고기 굽고, 가니쉬 올리고..
  비즈니스 로직 다 알고 있음.
- 근데 재료가 없음.

DAO(Data Access Object)= 창고 관리인

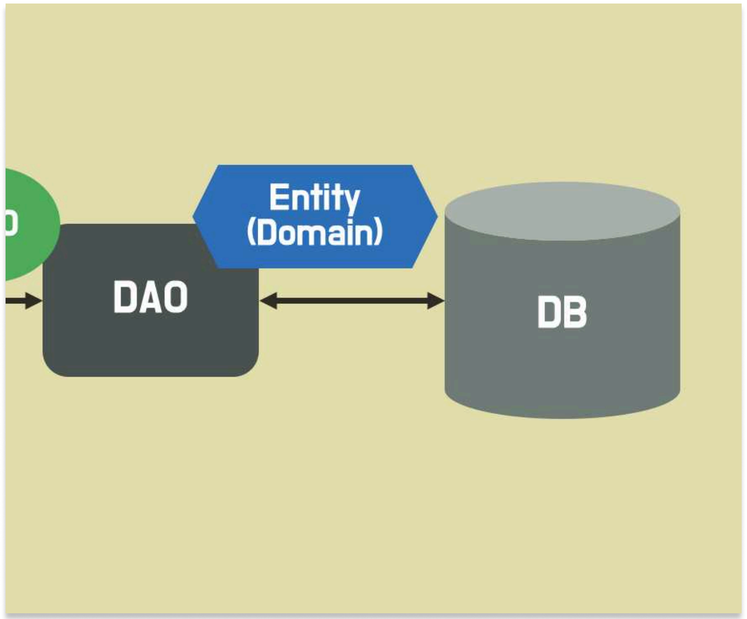
Service → DAO: 한우 등심 A+ 200g 내놔!!

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Layered Architecture = 레스토랑



DAO / Repository

- 창고 어느 선반에 한우 등심 있는지 위치 알고 있음.
- 재료 입고되면 정리까지 함.
- 출고도 잘 함.

DB = 냉장/냉동 창고

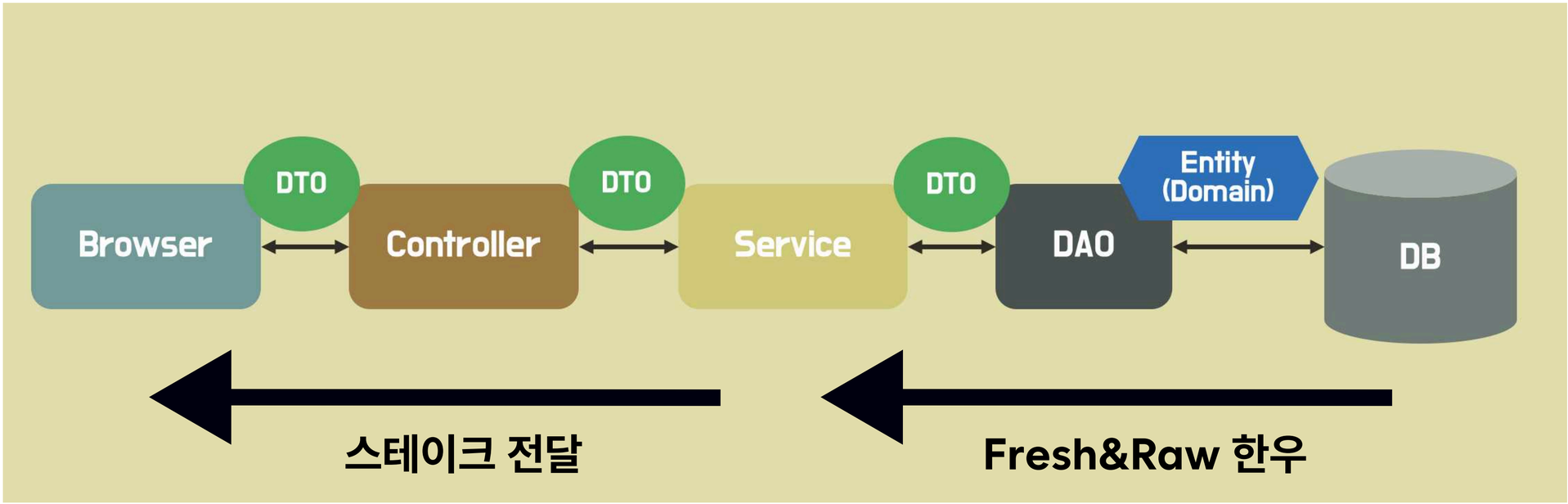
Repository→ DB: 한우 꺼내옴.

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Layered Architecture = 레스토랑

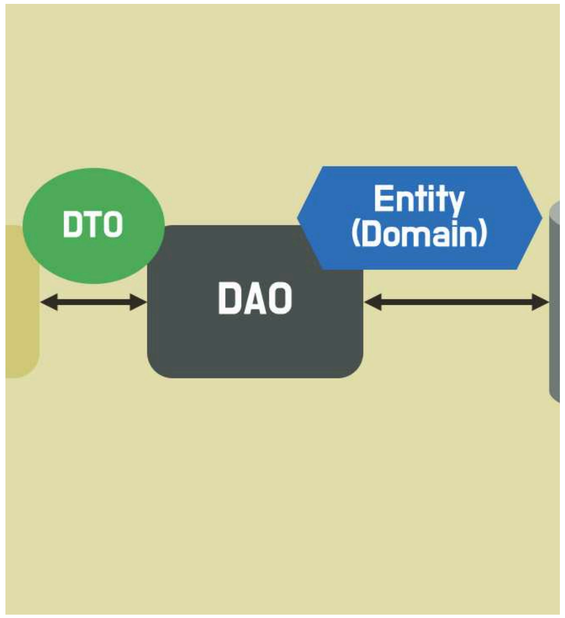


2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

DTO & Entity



DTO = 주문서,영수증

- 주문에 꼭 필요한 정보만 포함 ex. 메뉴 이름, 수량

- 데이터 전송 객체(Data Transfer Object)
- 소통 목적에 맞는, 필요한 정보만 전달

Entity = 원재료

- 모든 정보 포함 ex. 유통기한, 원산지, 등급

- DB 테이블과 맵핑되는 핵심 객체
- 외부 직접 노출 금지(데이터 일관성/보안)

2주차

Layered Architecture

스프링 계층형 아키텍처(Layered Architecture)?

Controller

Service

Repository

Entity

2주차

Layered Architecture

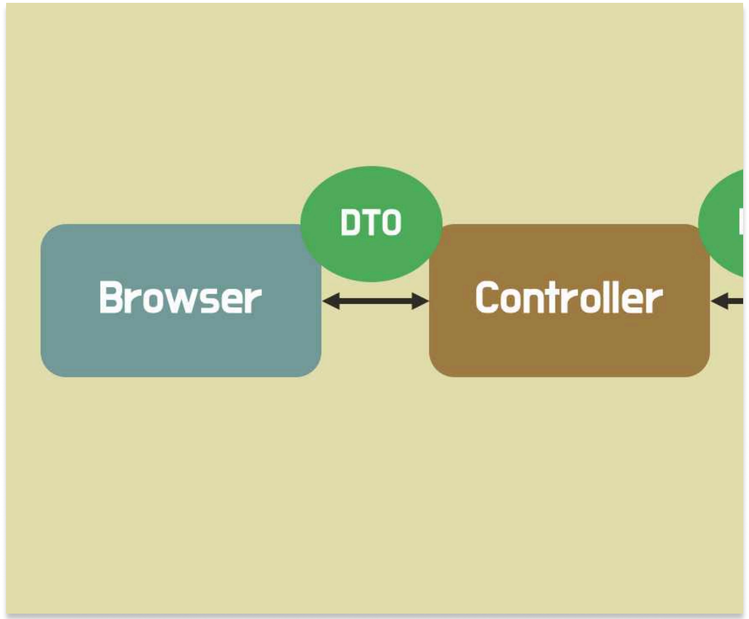
스프링 계층형 아키텍처(Layered Architecture)?

Controller Service

2주차

Controller Layer

컨트롤러 계층(Controller Layer)



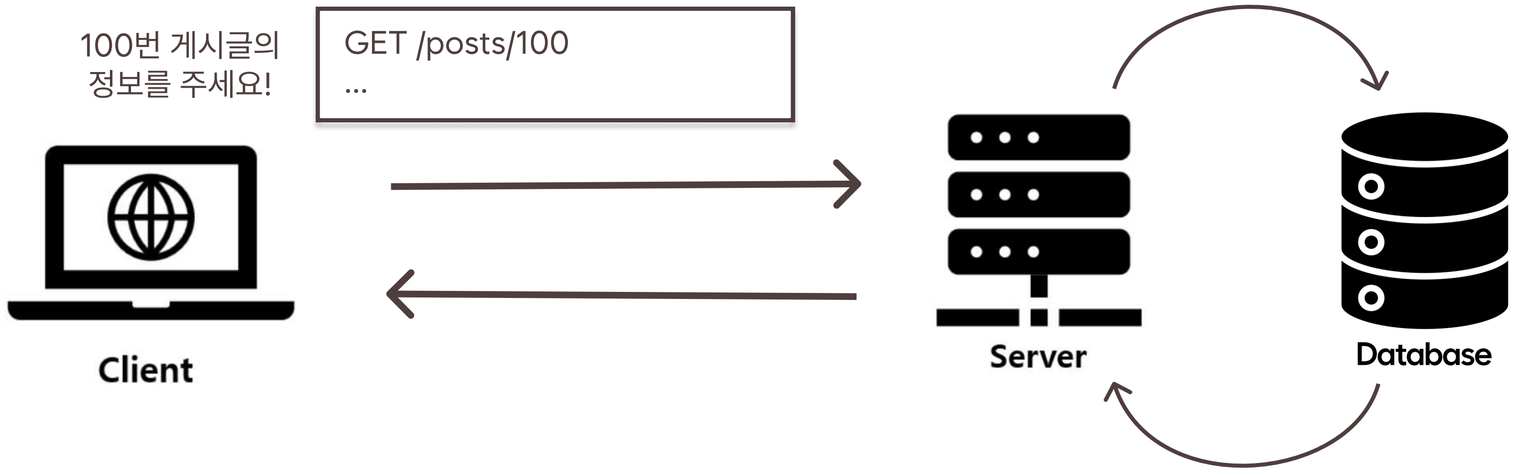
Controller = 웨이터

- HTTP 요청 / 응답
- 특정 endpoint(URL)로 요청을 가장 먼저 처리
- DTO (Data Transfer Object)를 사용하여 서비스 계층과 데이터 주고받음.

Review

프론트엔드와 백엔드

게시판 글 조회



Review

프론트엔드와 백엔드

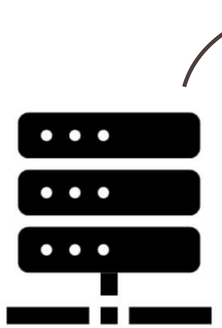
Front-end

게시판 글 조회

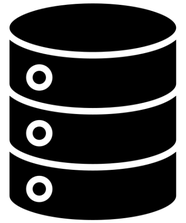
Client

화면 렌더링

Back-end



Server



Database

```
200 OK
...
{
   “title”: “물방울 작가 ...”,
   “content”: “제가 너무나...”
}
```

100번 게시글 정보입니다~

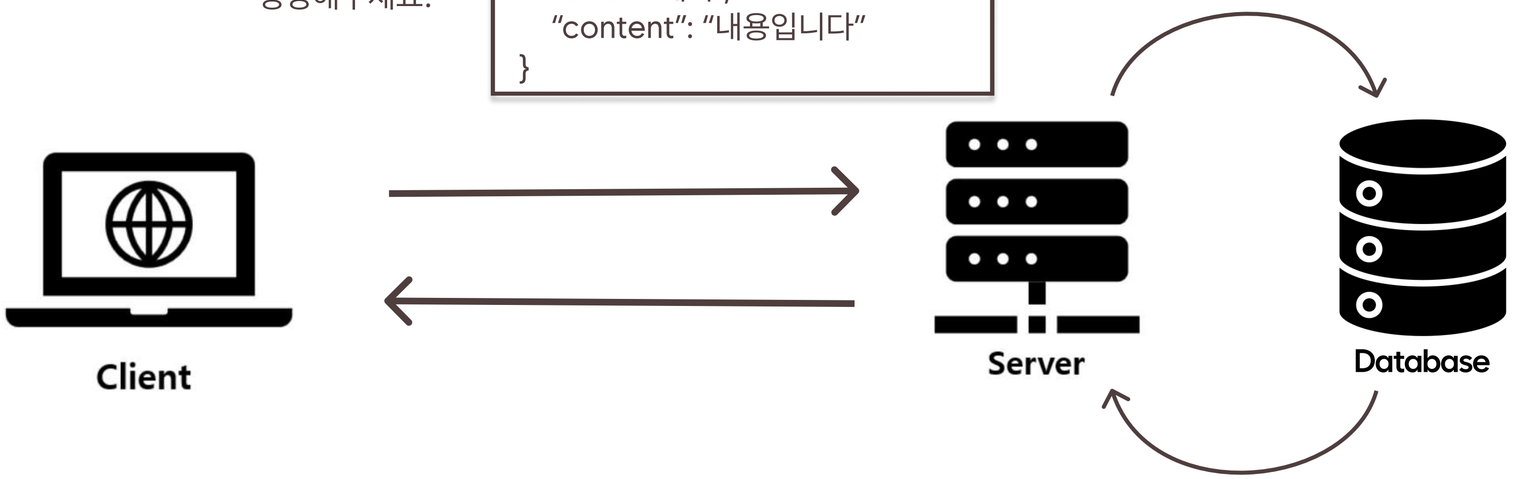
Review

프론트엔드와 백엔드

이 정보인 게시글을
생성해주세요!

```
POST /posts
...
{
    “title”: “제목”,
    “content”: “내용입니다”
}
```

게시판 글 작성

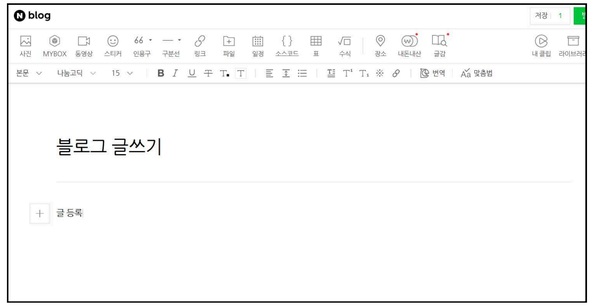


Review

프론트엔드와 백엔드

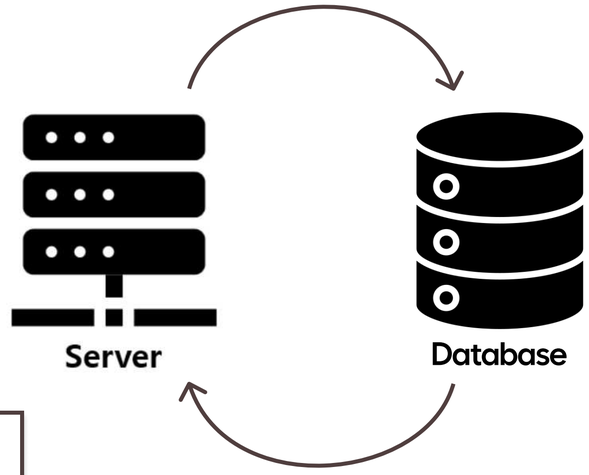
Front-end

Back-end



게시판 글 작성

Client



201 Created
...

생성에 성공했습니다~

Review

API와 REST API

RESTful하게 API 설계하기 - API 명세서 작성

요구 사항 - 온라인 쇼핑몰 프로젝트

1. 이름, 전화번호, 주소를 바탕으로 회원을 등록할 수 있다
2. 회원 리스트를 조회할 수 있다.
3. 회원 정보를 상세 조회할 수 있다.
4. 회원 정보를 수정할 수 있다.
5. 회원을 삭제할 수 있다.

Review

API와 REST API

RESTful하게 API 설계하기 - API 명세서 작성

1. 회원 등록

   HTTP Method : POST

   URI : /members

2. 회원 리스트 조회

   HTTP Method : GET

   URI : /members

Review

API와 REST API

RESTful하게 API 설계하기 - API 명세서 작성

3. 회원 상세 조회

HTTP Method : GET

URI : /members/{memberId}

4. 회원 정보 수정

HTTP Method : PATCH

URI : /members/{memberId}

5. 회원 삭제

HTTP Method : DELETE

URI : /members/{memberId}

path variable
URI 일부를 변수처럼 사용해서 특정 자원을 식별하는 방식

/members/1 1번 멤버만 지목
/members/2 2번 멤버만 지목
...

→ /members/{memberId}

2주차

Controller Layer

HTTP 데이터 구조

HTTP header

HTTP body

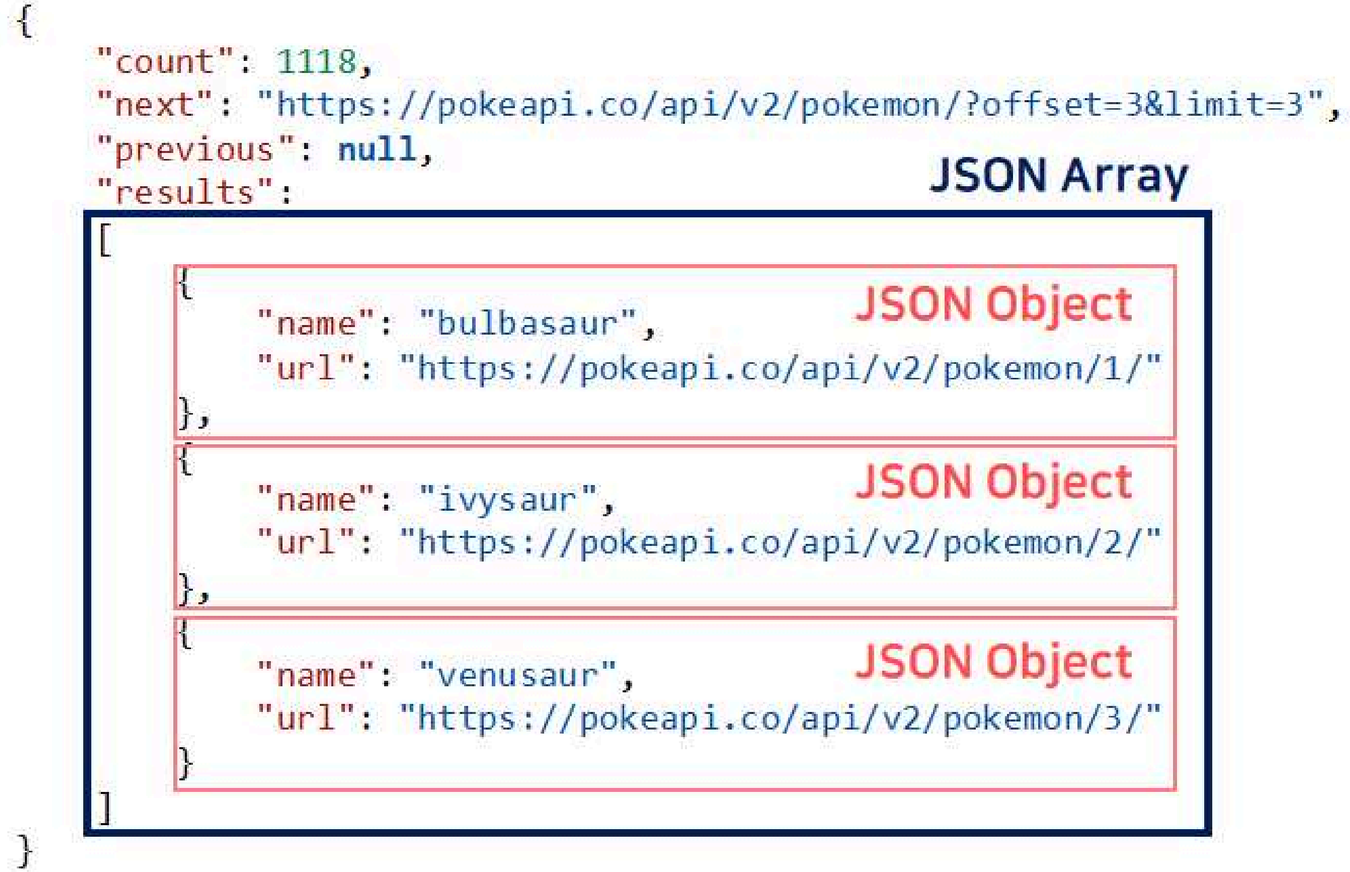
HTTP body
요청: request body
응답: response body

API 서버: json 응답

2주차

Controller Layer

JSON (JavaScript Object Notation)



2주차

Controller Layer

컨트롤러 만들어봅시다.

1. @Controller 어노테이션
2. @Responsebody 어노테이션
3. @RestController 어노테이션
4. 생성자 주입(서비스 계층에 의존)
5. @RequestMapping를 통해 method, url 지정.
6. 공통 URL & 상세 URL
7. @RequestBody를 통해 json 데이터 받아오기

2주차

Controller Layer

패키지 구조: 계층형 vs 도메인형

계층형 구조

애플리케이션을 기능별로 나눈다.
Controller는 Controller, Service는 service에!

```
com.example.shop
├── controller
│   ├── MemberController.java
│   ├── ProductController.java
│   └── OrderController.java
├── service
├── repository
├── domain
├── dto
├── config
└── exception
```

2주차

Controller Layer

패키지 구조: 계층형 vs 도메인형

도메인형 구조

도메인 관련 모든 클래스를 포함

- 특정 도메인의 코드를 한 곳에! -> 코드 탐색 easy
- 도메인 단위 개발, 유지보수하기 용이.
- New 도메인 추가 시 다른 곳에 영향 없음.

도메인형 구조로 갑시다.

2주차

Controller Layer

상태 코드(status code)

1. 200 OK [처리 성공]
2. 201 Created [데이터 생성 성공]
3. 204 No Content [요청 성공 but 보내줄 콘텐츠 없음] = Delete 통한 영구 삭제
4. 400 Bad Request [클라이언트 요청 오류]
5. 404 Not Found [요청 데이터 없음]
6. 500 Internal Server Error [서버 에러]

2주차

Controller Layer

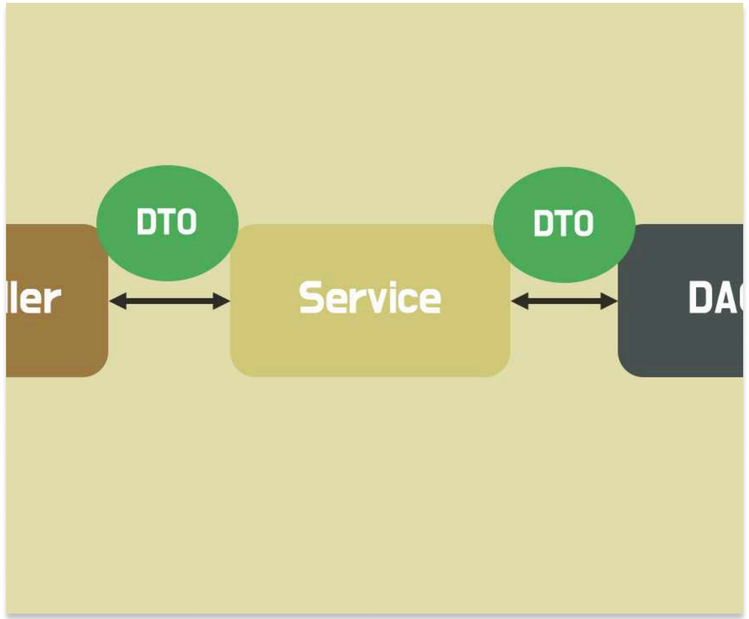
참고) ResponseEntity의 메서드

메서드	상태 코드	Body	사용 시점
`ok().build()`	200 OK	없음	성공, 데이터 불필요
`ok(data)`	200 OK	data	성공, 데이터 반환
`ok().body(data)`	200 OK	data	위와 동일 (명시적)
`created(uri).build()`	201 Created	없음	생성 성공
`noContent().build()`	204 No Content	없음	삭제 성공

2주차

Service Layer

서비스 계층(Service Layer)



Service = 요리사

- 애플리케이션의 비즈니스 로직이 담기는 계층
- 레포지토리 계층과 소통 with 엔티티 또는 DTO
- 컨트롤러와 레포지토리 사이의 중간 다리

2주차

Service Layer

서비스 계층: 비즈니스 로직

Ex. member 데이터 수정

1. member를 조회
2. member 데이터 수정

2주차

Service Layer

서비스 계층: 비즈니스 로직

Ex. member 데이터 수정

1. member를 조회 ← 오류 발생!!(존재하지 않음)
2. member 데이터 수정

2주차

Service Layer

서비스 계층: 비즈니스 로직

Ex. memberA_지름의_신() 메서드 실행

1. memberA가 갈치 10마리 구매
2. memberA가 iphone17 구매 ← 오류 발생!!!!!!
3. memberA가 샘숭 노트북 1대 구매

셋 다 실행 or 셋 다 실행 안 되거나.
더 이상 쪼갤 수 없음
원.자.성(atomicity)을 가진다.

2주차

Service Layer

서비스 계층: 비즈니스 로직

Ex. memberA_지름의_신() 메서드 실행

1. memberA가 갈치 10마리 구매
2. memberA가 iphone17 구매 ← 오류 발생!!!!!!
3. memberA가 샘숭 노트북 1대 구매

Sol) 트랜잭션 단위로 처리 for 원자성 보장.
→ 서비스 계층 메서드 위에 @transactional 붙이면 끝.

2주차

Service Layer

서비스 계층: 비즈니스 로직

트랜잭션.

BUT
2주차: JPA 사용 안함 + DB 대신 메모리 사용
@transactional 동작 안함 => 주석 처리

*JPA(Java Persistence API)는 자바의 ORM기술을 쉽게 구현하도록 도와주는 API이다.

2주차

Service Layer

서비스 계층 구현해봅시다.

1. 생성자 주입
2. @transactional [2주차: 주석 처리]
3. (readOnly = true) 옵션

→ 조회만 하는 경우, 트랜잭션 내에서 데이터가 변경되지 않도록 readOnly 속성을 활성화

2주차

스프링 빈 & 의존성 주입

스프링? 스프링 부트?

스프링(Spring)

- 대표적인 Java 백엔드 애플리케이션 프레임워크
- 객체지향 원리를 잘 지키게 해줌.

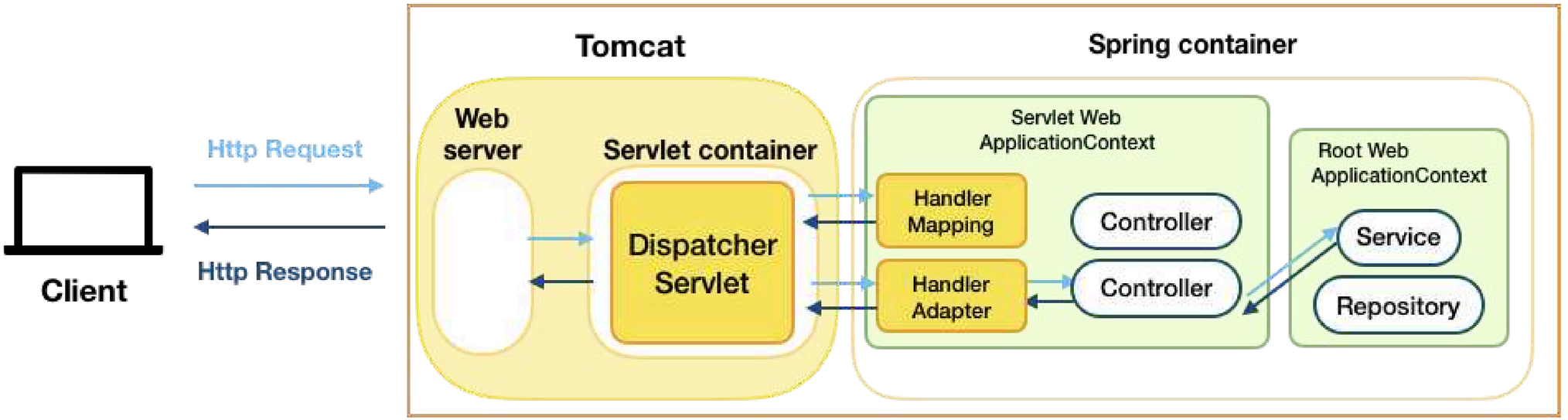
스프링 부트(Spring Boot)

- 스프링 프레임워크를 쉽고 편하게 쓸 수 있게 돕는 도구
- 스프링 짝꿍.

2주차

스프링 빈 & 의존성 주입

스프링 애플리케이션 구조



2주차

스프링 빈 & 의존성 주입

빈 & 스프링 컨테이너

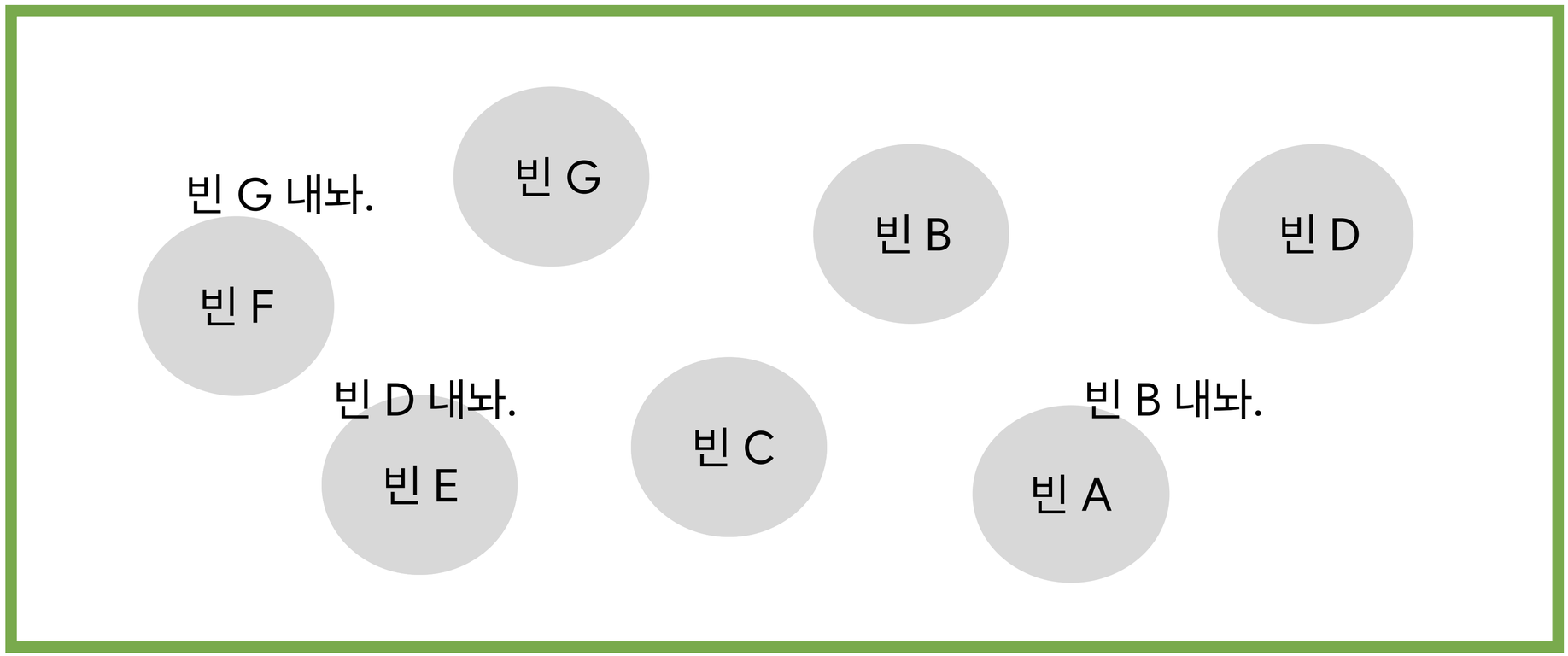
Spring Container

- 스프링 빈 저장소
- 어플리케이션 컨텍스트(Application Context)

2주차

스프링 빈 & 의존성 주입

빈 & 스프링 컨테이너



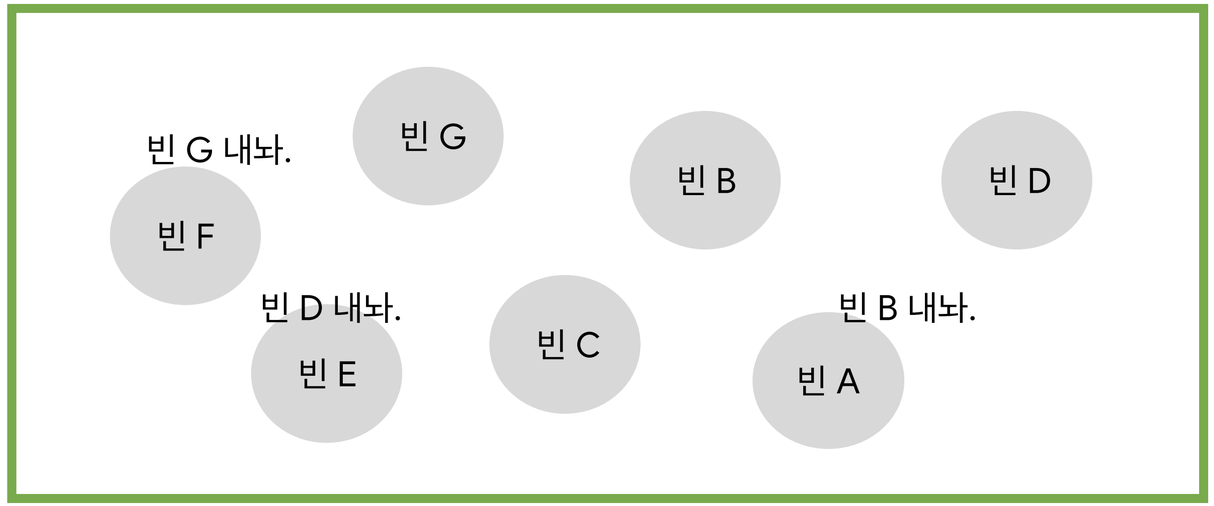
2주차

스프링 빈 & 의존성 주입

빈 & 스프링 컨테이너

스프링 빈 (Spring Bean)

- 어플리케이션 전역에서 사용할 **공용 객체**
- 스프링 컨테이너(공용 창고)에 빈을 저장, 필요한 빈을 컨테이너에서 받아 사용한다.



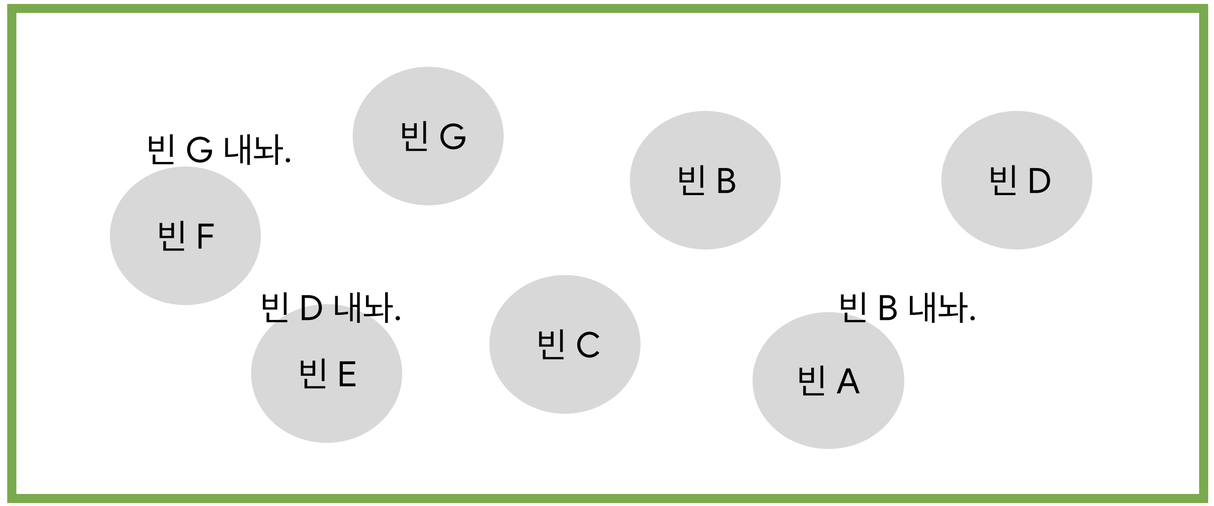
2주차

스프링 빈 & 의존성 주입

빈 & 스프링 컨테이너

스프링 빈 (Spring Bean)

- 필요한 빈은 스프링 프레임워크가 자동으로 가져다 준다.
- 빈을 요구하는 객체 역시 스프링 빈이다.



2주차

스프링 빈 & 의존성 주입

빈 등록 방법

- 설정 파일 작성 (수동 등록)
- 컴포넌트 스캔 (자동 등록) ✅

2주차

스프링 빈 & 의존성 주입

빈 등록 방법: 컴포넌트 스캔 (자동 등록)

@Component, @ComponentScan

1. 컴포넌트 스캔 (@ComponentScan)
   - "어떤 클래스들이 Spring Bean인지 찾아서 등록"

2. 컴포넌트 (@Component)
   - "이 클래스가 Spring Bean이야!"라고 표시

→ 빈으로 등록하고 싶은 클래스에 @Component 붙이면 된다.

2주차

스프링 빈 & 의존성 주입

이게 뭡니까?

1. @Controller
2. @RestController
3. @Service
4. @Repository
5. @SpringBootApplication

.

.

.

2주차

스프링 빈 & 의존성 주입

컴포넌트 스캔(자동 등록)

1. @Controller
2. @RestController
3. @Service
4. @Repository
5. @SpringBootApplication

.
.
.

1. 컴포넌트 스캔 (@ComponentScan)

- @SpringBootApplication에 포함됨
- "어떤 클래스들이 Spring Bean인지 찾아서 등록"

2. 컴포넌트 (@Component)

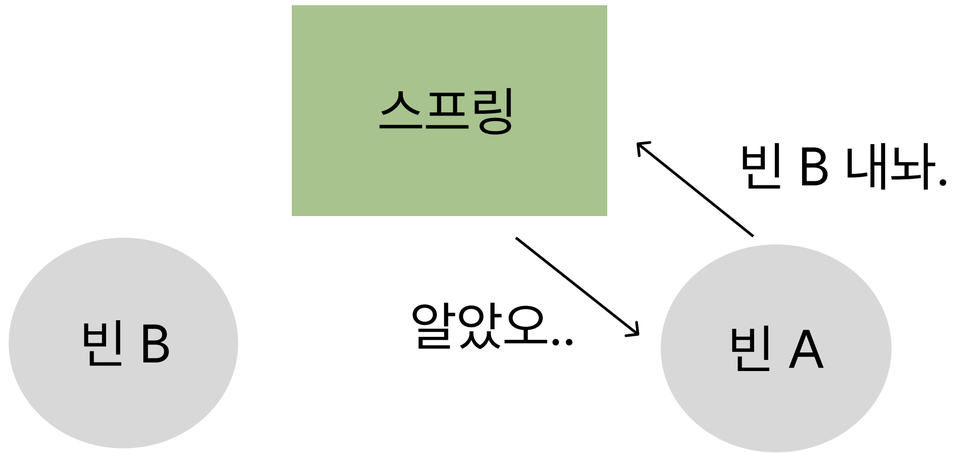
- @Controller, @Service, @Repository @Entity 등에 포함됨.
- "이 클래스가 Spring Bean이야!"라고 표시

2주차

스프링 빈 & 의존성 주입

의존성 주입(Dependency Injection, DI)

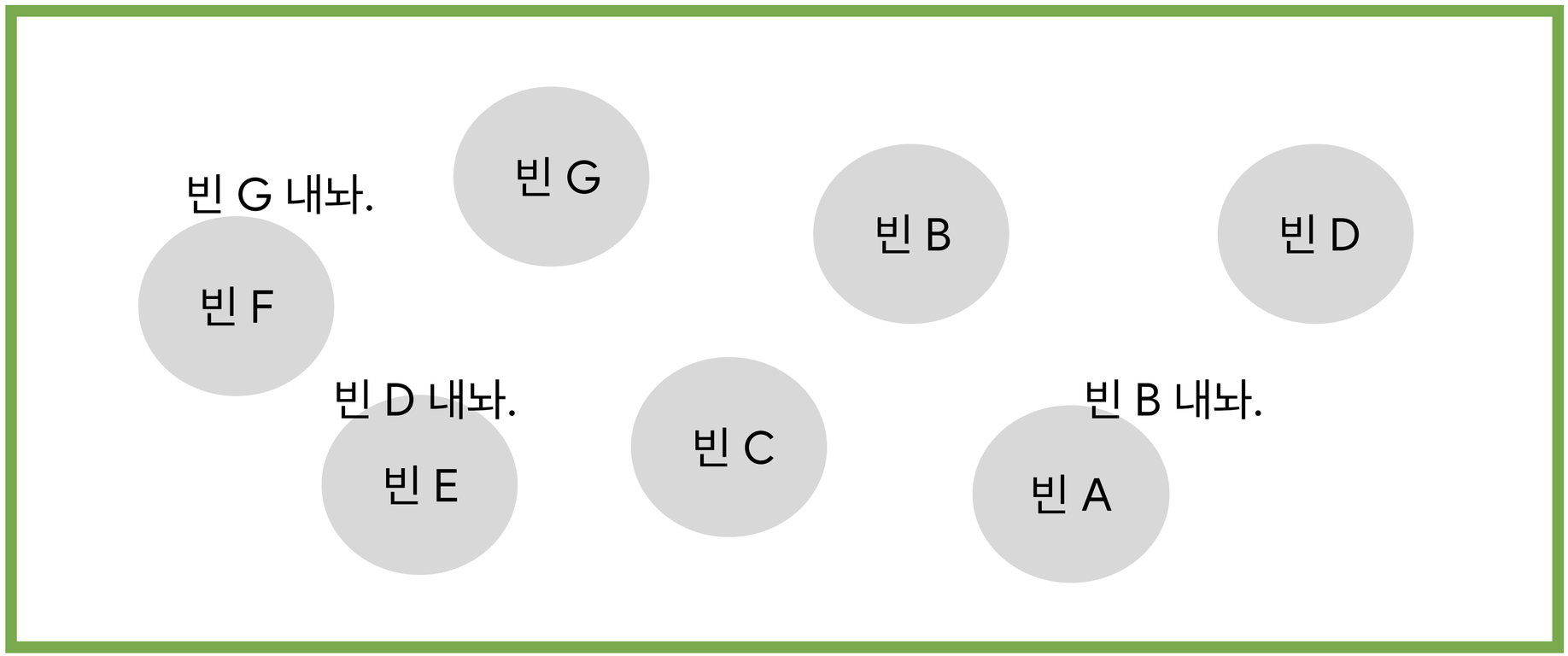
내가 의존하는 객체를 직접 생성하지 않고 밖에서 주입 받는 것



2주차

스프링 빈 & 의존성 주입

의존성 주입(Dependency Injection, DI)



2주차

스프링 빈 & 의존성 주입

의존성 주입 왜 써?

스프링 컨테이너에 있는 거 쓰는 게 좋아서?

매번 필요한 객체를 생성하는 대신,
생성해둔 객체를 사용하므로
메모리를 효율적으로 사용

2주차

스프링 빈 & 의존성 주입

의존성 주입 방법

1. **생성자 주입**
2. 필드 주입
3. 수정자 주입 (세터 주입)

2주차

스프링 빈 & 의존성 주입

의존성 주입 방법

1. **생성자 주입**
2. ~~필드 주입~~
3. ~~수정자 주입 (세터 주입)~~

2주차

스프링 빈 & 의존성 주입

생성자 주입 방법

1. 안전하게 final로 선언
   [final: 변수에 한 번만 값을 할당할 수 있게 하여 변경을 막음.]
2. 생성자에 @Autowired을 사용하면, 생성자를 통해 빈을 주입한다.
3. 만약 생성자가 하나만 있다면, @Autowired를 생략할 수 있다.

2주차

스프링 빈 & 의존성 주입

생성자 주입 방법(간단ver)

1. 필요한 의존성을 final 키워드를 사용해 추가.
2. @RequiredArgsConstructor를 사용해 생성자를 추가한다.

과제

Homework

온라인 쇼핑몰 프로젝트(어드민)

1. Member, Product, Order에 대한 컨트롤러 & 서비스 계층 구현
   레포지토리, 엔티티 X
2. 강의 내용 요약 정리하기(wil.md)

마무리

끝